Proving Data Structures Correct: Abstract Data Types

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Where We Are in the Course

- **Covered in Weeks 1–7:**
  - **Week 1:** ITP fundamentals and the Isabelle/HOL environment.
  - **Week 2:** Mathematical induction and hypothesis generalisation.
  - **Week 3:** List recursion, structural induction, and proof automation.
  - **Week 4:** Algebraic datatypes, tree induction, binary search trees.
  - **Week 5:** AI-assisted proving (Sledgehammer, coding agents via MCP).
  - **Week 6:** Correctness of sorting, computation induction, running times.
  - **Week 7:** Balanced search trees: 2-3 trees, their height, and insertion.
- **Week 8:** Abstract data types: specified by locales, implemented by search trees.
- **Next, Week 9:** Graph algorithms: depth-first search.

©Mohammad Abdulaziz

Outline

1. Specifying ADTs
   - Interface and specification, abstraction function and invariant
   - Locales in Isabelle
2. Using ADTs
   - Generic algorithms inside a locale
3. Implementing ADTs
   - Interpretation: Week 4's binary search trees and Week 7's 2-3 trees

©Mohammad Abdulaziz

Abstract Data Types

A methodological interlude:

A closer look at ADT principles
and their realisation in Isabelle

Set as example, implemented by
binary search trees and 2-3 trees
(ignoring `delete`)

©Mohammad Abdulaziz

Defining ADTs

ADT = *interface* + *specification*

©Mohammad Abdulaziz

Example (Set interface)

```
empty :: 's
insert :: 'a ⇒ 's ⇒ 's
isin :: 's ⇒ 'a ⇒ bool
```

We assume that each ADT describes one

Type of Interest T

Above: $T =$'s

Model-oriented specification

Specify type T via a model = existing HOL type A

Motto: T should behave like A

Specification of "behaves like" via an

- *abstraction function* $\alpha :: T \Rightarrow A$

Only some elements of T represent elements of A:

- *invariant* invar :: $T \Rightarrow$ bool

α and invar are part of the interface,
but only for specification and verification purposes

©Mohammad Abdulaziz

Example (Set ADT)

```
empty :: ...
insert :: ...
isin :: ...
set ::    's ⇒ 'a set   (name arbitrary)
invar ::  's ⇒ bool     (name arbitrary)

                set empty = {}
 invar s ⟹      set(insert x s) = set s ∪ {x}
 invar s ⟹      isin s x = (x ∈ set s)
                invar empty
 invar s ⟹      invar(insert x s)
```

In Isabelle: **locale**

```
locale Set =
fixes empty :: 's
fixes insert :: 'a ⇒ 's ⇒ 's
fixes isin :: 's ⇒ 'a ⇒ bool
fixes set :: 's ⇒ 'a set
fixes invar :: 's ⇒ bool
assumes set empty = {}
assumes invar s ⟹ isin s x = (x ∈ set s)
assumes invar s ⟹ set(insert x s) = set s ∪ {x}
assumes invar empty
assumes invar s ⟹ invar(insert x s)
```

See `HOL/Data_Structures/Set_Specs.thy`

Formally, in general

To ease notation, generalise α and invar (conceptually):

α is the identity and invar is `True`
on types other than T

Specification of each interface function f (on T):

- f must behave like some function f_A (on A):
  $\text{invar}\ t_1 \wedge \cdots \wedge \text{invar}\ t_n \implies$
  $\alpha(f\ t_1 \ldots t_n) = f_A(\alpha\ t_1) \ldots (\alpha\ t_n)$

  (α is a homomorphism)
- f must preserve the invariant:
  $\text{invar}\ t_1 \wedge \cdots \wedge \text{invar}\ t_n \implies \text{invar}(f\ t_1 \ldots t_n)$

©Mohammad Abdulaziz